

Vibe Coding 101

Membangun perangkat lunak nyata bersama agent — dari menyiapkan lingkungan sampai ngoding bareng.

- 01** **Sharing pengalaman**
cerita dari proyek nyata.
- 02** **Mengenal Claude Code environment**
CLAUDE.md, skills, MCP, hooks.
- 03** **Prompt project & review**
menulis prompt tepat & bedah hasilnya.
- 04** **Note taking & tanya jawab**
catat insight & diskusi.
- 05** **Let's Get Vibing**
ngoding langsung, bareng.





— TUTORIAL · VIBE CODING

Membangun Dashboard bersama Claude Code

Studi kasus nyata: dashboard **coverage SPPG** — membandingkan jumlah dapur Makan Bergizi Gratis dengan populasi penerima (peserta didik PAUD & SD) di tiap kecamatan se-Indonesia.

24.238

DAPUR SPPG

4.948

KECAMATAN

38

PROVINSI

~90%

SELESAI

Sebelum agent menulis kode: selaraskan arah

Agent sehebat konteks yang kita beri. Pertama kali, developer tidak langsung "menyuruh" — tapi menyiapkan tiga hal agar hasilnya terarah, bukan menebak.

01 Objektif yang ingin dicapai

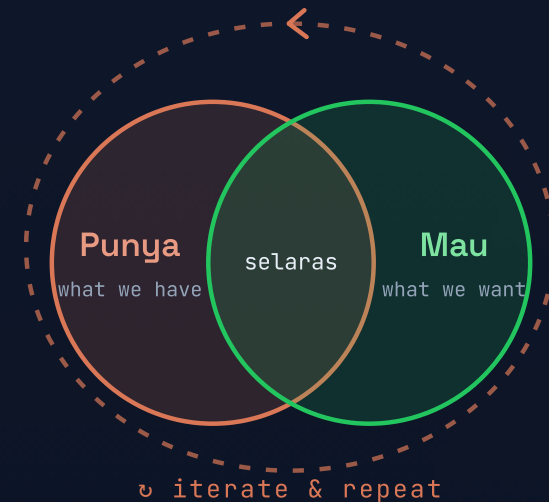
Definisikan hasil akhir & batasannya. Target kabur = agent mengisi asumsinya sendiri.

02 Knowledge & materi yang sudah dimiliki

Inventaris yang ada: data, dokumen, kode lama, standar, contoh. Inilah bahan konteks — CLAUDE.md, docs, skills.

03 Selaraskan "what we have" & "what we want"

Jembatani keduanya: prompt → tinjau → koreksi. **iterate & repeat** sampai irisannya penuh.



Tiap iterasi memperbesar irisan — jarak antara yang dipunya dan yang diinginkan mengecil.

Struktur yang membuat agent paham proyek

Bukan sekadar folder kode. Tiap bagian memberi konteks ke Claude: di mana logika, di mana data, di mana aturan.

data-bgn/

KONTEKS UNTUK AGENT

└─ CLAUDE.md	Peta proyek + aturan domain. Dibaca agent tiap sesi.
└─ .mcp.json	Daftar server MCP (code-review-graph).
└─ .claude/	Konfigurasi khusus Claude Code.
└─ └─ skills/	4 skill spesifik repo (cara kerja tiap lapisan).
└─ └─ settings.json	Hooks: auto-update graph + diagnostic gate.

APLIKASI

└─ backend/	API Express + pg, rumus coverage.
└─ frontend/	Dashboard Vue 3 + Vite (tabel, peta).
└─ infrastructure/	docker-compose (Postgres) + nginx.
└─ script/	Pipeline data: scrape → clean → SQL.

PENGETAHUAN & MATERI

└─ docs/	Standar kode, SRS, proses bisnis, ERD + flow (mermaid).
└─ presentation/	Slide deck ini.

- **Pemisahan jelas.** Backend, frontend, infrastruktur, dan data hidup di rumahnya masing-masing.
- **Dapat dijalankan.** Struktur ini bukan teori — DB, API, dan dashboard benar-benar menyala.
- **Self-documenting.** Folder atas = konteks agent, tengah = kode, bawah = pengetahuan.

CLAUDE.md & Skills — memori & keahlian

• CLAUDE.md

Dibaca tiap sesi. Berisi peta folder, perintah dev, alur data, dan **aturan domain** yang tak bisa ditebak dari kode saja.

```
# aturan yang "dikunci"  
PORSI_PER_SPPG = 2000  
tier: <70 • 70-89 • 90-100 • >100  
selalu query dari v_sppg (38 prov bersih)
```

• 4 Skill proyek

Pengetahuan spesifik repo, dipanggil saat relevan:

- **node-express-api** — pola endpoint, try/catch, view bersih.
- **vue-dashboard-ui** — halaman, peta, warna tier.
- **sppg-data-pipeline** — scrape → clean → import.
- **db-migrations** — skema & view idempoten.

Tools & disiplin: MCP + Diagnostic Gate

• code-review-graph (MCP)

Tree-sitter → graph SQLite dari seluruh kode. Claude membaca **hanya yang relevan** + cek blast-radius sebelum mengubah fungsi.

132

NODE GRAPH

902

EDGE RELASI

• Diagnostic Gate

Hook otomatis tiap kali kode berubah — agent diingatkan menjalankan urutan ini **sebelum** tes browser:

- 1 LSP diagnostics — perbaiki error bahasa.
- 2 Unit test — `node --test` harus hijau.
- 3 Tes browser — Playwright / Chrome.

01 Fondasi: database yang sudah ada

Kita mulai dari data nyata di PostgreSQL — 24k SPPG, 7k kecamatan DAPODIK, tabel pemetaan & koordinat. Tugas pertama: pahami skema, lalu bersihkan.

prompt ke Claude

```
> Pelajari skema database sppg di Postgres :5435. Buat view bersih yang hanya memuat 38 provinsi valid & perbaiki schema drift.
```

```
# hasil
```

```
v_sppg · ref_provinsi · coords ditambah  
provinsi 61 → 38
```

terminal · psql

```
> docker exec -it sppg-db psql -U postgres -d sppg
```

```
sppg=# \dt  
public | sppg | table  
public | dapodik_pd | table  
public | kecamatan_mapping | table  
public | kecamatan_coords | table  
public | ref_provinsi | table  
(6 rows)
```

```
> SELECT count(*) FROM sppg;  
24264 -- mentah (ada provinsi kotor)
```

```
> SELECT count(*) FROM v_sppg;  
24238 -- view bersih
```

```
> SELECT count(DISTINCT provinsi) FROM v_sppg;  
38 ✓
```

02 Backend API & logika coverage

Express + pg menyajikan `/api/*`. Inti bisnis — kapasitas, persentase, tier — jadi satu sumber-kebenaran yang diuji unit test.

prompt ke Claude

```
> Hitung coverage = (jumlah_sppg × 2000) / penerima, tandai tier merah/kuning/hijau. Taruh rumus di satu modul + unit test.
```

```
# node --test
```

```
✓ 6 pass · 0 fail coverage.test.js
```

terminal · node --test

```
> cd backend && node --test
```

```
✓ kapasitas = jumlah SPPG × porsi  
✓ coveragePct: penerima tidak valid → null  
✓ coveragePct: rumus benar  
✓ tier: 4 tingkat merah/kuning/h_muda/h_tua  
✓ tier: tepat di batas (70 / 90 / 100)  
✓ tier: tanpa data penerima → null
```

```
i tests 6
```

```
i pass 6 i fail 0
```


03 Dashboard Vue & peta sebaran

Vue 3 + Vite + Tailwind. Tabel, ringkasan, dan peta Leaflet yang mewarnai tiap kecamatan sesuai tier coverage — bahasa visual yang sama dari ujung ke ujung.

prompt ke Claude

```
> Buat halaman peta Leaflet; warnai kecamatan sesuai tier coverage & tangani lat/lon kosong.
```

```
# vite build
```

```
✓ built 1583 modules
```

Dashboard Monitoring
Dapur SPPG
Kementerian Pendidikan

Overview

Peta Distribusi

Data SPPG

Peserta Didik

Analisis Coverage

Laporan

v1.0.0 - Prototype

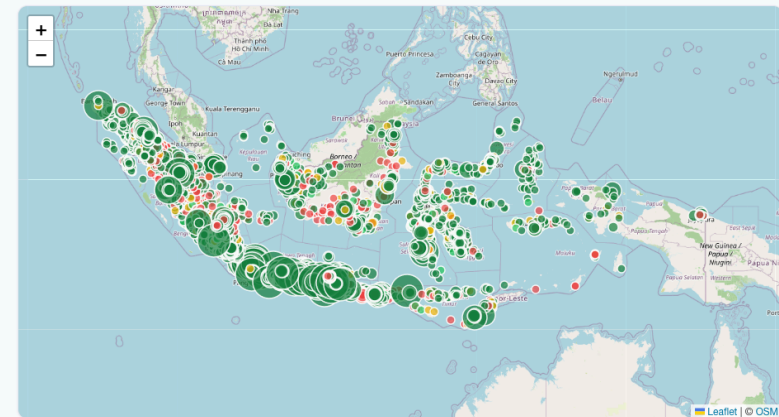
Peta Distribusi SPPG

Legenda: ● Hijau tua (>100%) ● Hijau muda (90-100%) ● Kuning (70-89%) ● Merah (<70%) ● Tidak ada data PD

Total: 4948 kecamatan

Simulasi kapasitas 1 SPPG = 2.000 porsi/hari

Hasil: ▲ 3606 ● 194 ● 375 ● 772 N/A 1 Rata-rata coverage: 169.3%



Peta Distribusi — warna kecamatan ikut tier coverage

04 Loop kualitas: diagnose → uji → lacak

Tiap perubahan kode melewati gerbang yang sama. code-review-graph melacak dampaknya. Inilah yang menjaga proyek tetap presisi & minim bug.

● Tulis

Edit oleh Claude. Hook PostToolUse aktif.

● Diagnose

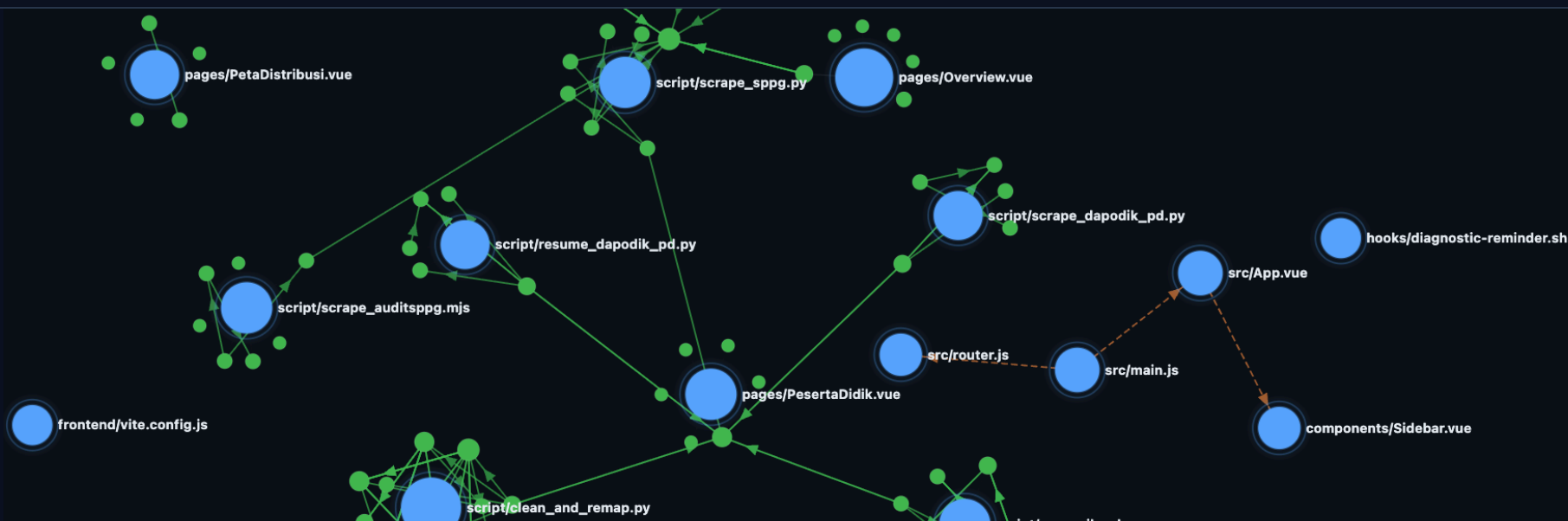
LSP cek bahasa — error nyata diperbaiki.

● Uji

`node --test` + smoke test endpoint.

● Lacak

graph update; cek di browser bila lolos.



05 Prompt explorer: konteks & clarity

Tidak ada satu cara benar. Terus belajar menakar **seberapa banyak konteks** yang dibutuhkan agent — disesuaikan dengan bobot tugas. Tiga model prompt yang sering dipakai:

● 1 • Lengkap & presisi



```
# Objektif
hasil akhir + batasan
# Peran
"kamu expert di bidang X"
# Tahapan
langkah presisi 1→n
# Rules & Larangan
yang tak boleh dilakukan
```

Kapan: tugas besar, baru, kompleks, berisiko.

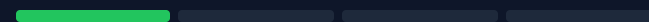
● 2 • Menengah



```
> Bangun X yang melakukan Y.
1. tahap pertama
2. tahap kedua
3. hal yang perlu diperhatikan
```

Kapan: arah jelas, lingkup sedang, pola dikenal.

● 3 • Ringkas



```
> Perbaiki bug di /sppg.
> Tambah tombol export.
> Warnai header biru.
```

Kapan: fix, fitur kecil, styling sederhana.

↳ Makin besar bobot & risiko tugas → makin banyak konteks & rules yang perlu diberi.

Yang membuat vibe coding berhasil

- **Konteks dulu, kode kemudian.** CLAUDE.md + skills membuat agent paham aturan domain.
- **Disiplin terotomasi.** Diagnostic gate & unit test menangkap bug lebih awal.
- **Struktur yang jujur.** Pemisahan folder mencerminkan tanggung jawab nyata.

Catatan untuk diisi

Ruang untuk poin diskusi Anda: pelajaran, tantangan data, atau langkah berikutnya proyek SPPG.





— PENUTUP · 2 DARI 2

Diskusi & Tanya Jawab

Mari telusuri kode bersama — backend, frontend, atau pipeline data. Semua siap dijalankan langsung dari folder proyek.

mulai demo

```
# database
```

```
> cd infrastructure && docker compose up -d
```

```
# backend & frontend
```

```
> cd backend && npm run dev
```

```
> cd frontend && npm run dev
```